

Linear and Logistic Regression in Machine Learning

PAGE 1

A beginner's 3-page visual guide — no maths background needed

Concepts & Linear Regression

JUMP TO:

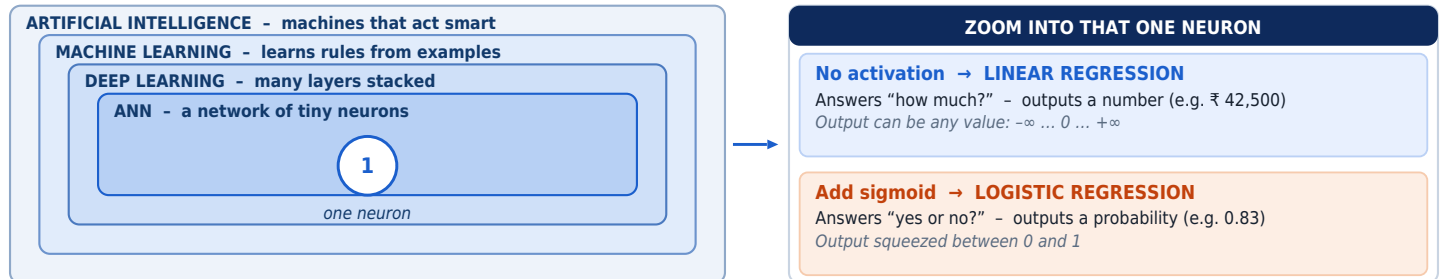
1. Big picture + Linear Regression

2. Training loop + Logistic Regression

3. Confusions, overfitting, comparison

1 The big picture — who lives inside whom?

Machine Learning (ML) is a way of programming where you do *not* write the rule yourself. You show the computer many solved examples and it works out the rule itself — like teaching a child what an apple is by pointing at many apples instead of giving a rulebook. That learned rule is stored as a set of numbers called **weights**. Everything below is this same idea at different sizes.



✓ REMEMBER

Linear and Logistic Regression are not separate from Deep Learning — they **are** the single building block that Deep Learning stacks thousands of times. Learn these two well and a neural network becomes easy.

2 Linear Regression — predicting a number

Purpose: draw the best straight line through your data so you can predict a **continuous number** for a new, unseen case. "Best" means the line whose predictions sit as close as possible to the real answers.

Real-life question	Inputs (x)	Output (y)
What price for this flat?	size, bedrooms, age	₹ price
What salary can I expect?	experience, skills, city	₹ salary
How long will delivery take?	distance, traffic, weight	minutes
What will my power bill be?	units, season, tariff	₹ amount

SIMPLE vs MULTIPLE

SIMPLE — one input

$$\hat{y} = w_1x_1 + b \quad \text{e.g. price from size only}$$

MULTIPLE — many inputs

$$\hat{y} = w_1x_1 + \dots + w_nx_n + b \quad \text{price from 3 things}$$

Same maths — only the number of terms changes.

x input feature — a fact you know

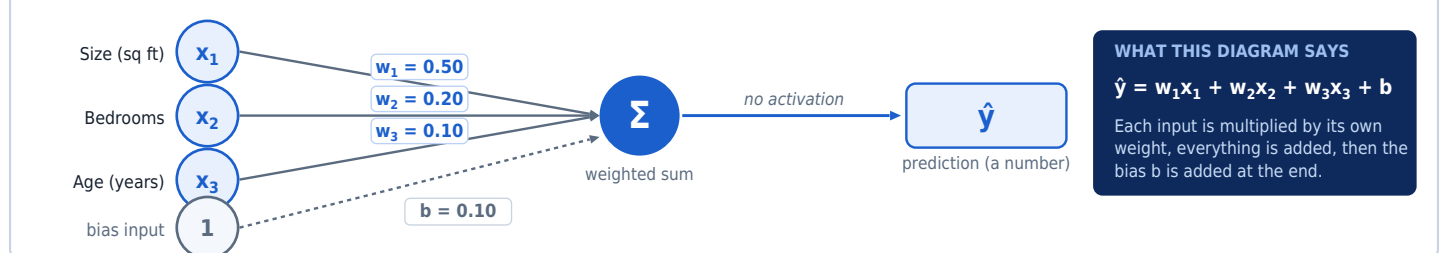
w weight — how much that fact matters

b bias — the starting value when all $x = 0$

$\hat{y}$ prediction — the model's answer

y target — the true, real answer

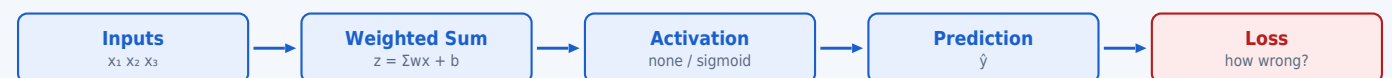
NETWORK DIAGRAM — multiple linear regression with 3 input features



3 How the model learns — guess, measure, correct, repeat

The weights above did not appear by magic. They start as random (or zero) numbers and get corrected again and again. Every round has two halves: a **forward pass** that guesses, and a **backward pass** that uses the mistake to improve.

FORWARD PASS — make a guess and measure how wrong it is



BACKWARD PASS — use the mistake to improve every weight



1 FORWARD

$$\hat{y} = w \cdot x + b$$

Multiply, add, guess.

2 LOSS

$$\text{MSE} = (\hat{y} - y)^2$$

Turn the mistake into one score.

3 GRADIENT

$$\partial L / \partial w = 2(\hat{y} - y) \cdot x$$

Slope: which way is downhill?

4 BACKPROP

chain rule

Send that slope back to every w .

5 UPDATE

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Step downhill. Repeat.

JUMP TO:

1. Big picture + Linear Regression

2. Training loop + Logistic Regression

3. Confusions, overfitting, comparison

4 Linear Regression by numbers — two passes of the loop

► THE SETUP

Using the network from page 1: a flat with $x_1=1$, $x_2=2$, $x_3=3$. The model currently guesses $\hat{y} = (0.50 \times 1) + (0.20 \times 2) + (0.10 \times 3) + 0.10 = 1.30$, but the true answer is **2.00**. Watch it close that gap. Follow one row at a time — every number below is calculated from the row above it.

WORKED EXAMPLE — LINEAR REGRESSION

inputs $x = (1, 2, 3)$ • true answer $y = 2.00$ • learning rate $\eta = 0.01$

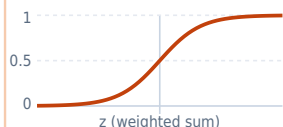
Step	Pass 1	Pass 2
Weights & bias at start	$w = 0.500, 0.200, 0.100$ $b = 0.100$	$w = 0.514, 0.228, 0.142$ $b = 0.114$
Forward: $\hat{y} = w \cdot x + b$	1.300	1.510
Error $e = \hat{y} - y$	-0.700	-0.490
Loss $MSE = e^2$	0.4900	0.2401 ▼ smaller
Gradient $\partial L / \partial \hat{y} = 2e$	-1.400	-0.980
Gradients $\partial L / \partial w = 2e \cdot x$	-1.400, -2.800, -4.200	-0.980, -1.960, -2.940
Gradient $\partial L / \partial b = 2e$	-1.400	-0.980
Update $w \leftarrow w - \eta \cdot \text{grad}$	0.514, 0.228, 0.142	0.5238, 0.2476, 0.1714
Update $b \leftarrow b - \eta \cdot \text{grad}$	0.114	0.1238

Gradients are negative, so subtracting them **increases** the weights — the prediction moves up toward 2.00 and the loss almost halves in a single step. That is learning.

5 Logistic Regression — predicting yes or no

Same weighted sum as before, but the question changes from “**how much?**” to “**which class?**” — spam or not spam, pass or fail, will buy or will not buy. This is called **binary classification**. The model never says a flat yes; it reports a **probability**, and you convert that into a decision with a threshold.

THE SIGMOID — turns any number into a probability



The weighted sum z can be any number (-100 or $+85$). A probability must sit between **0 and 1**. Sigmoid squeezes it: big positive $z \rightarrow$ near 1, big negative $z \rightarrow$ near 0, $z = 0 \rightarrow$ exactly 0.5. Then a **threshold** decides the class: $\hat{y} \geq 0.5 \rightarrow$ **Yes (1)**, otherwise **No (0)**.

$$\sigma(z) = 1 / (1 + e^{-z})$$

NETWORK DIAGRAM — logistic regression with 3 input features (example: will this customer buy?)



LOSS (BCE): $L = -[y \cdot \ln(\hat{y}) + (1-y) \cdot \ln(1-\hat{y})]$

If $y = 1$ only the first half counts $\rightarrow L = -\ln(\hat{y})$. Confident-and-wrong is punished very hard.

TWO COMPLETE PASSES — LOGISTIC REGRESSION

 $x = (2, 1, 3)$ • true $y = 1$ (did buy) • all weights start at 0 • $\eta = 0.1$

Step	Pass 1	Pass 2
Weights & bias at start	$w = 0, 0, 0$ $b = 0$	$w = 0.100, 0.050, 0.150$ $b = 0.050$
1. Weighted sum z	$0 \times 2 + 0 \times 1 + 0 \times 3 + 0 = 0.000$	$0.1 \times 2 + 0.05 \times 1 + 0.15 \times 3 + 0.05 = 0.750$
2. Sigmoid $\hat{y} = \sigma(z)$	0.5000	0.6792
3. BCE loss $= -\ln(\hat{y})$	0.6931	0.3869 ▼ smaller
4. Signal $dz = \hat{y} - y$	$0.5 - 1 = -0.5000$	$0.6792 - 1 = -0.3208$
5. $\partial L / \partial w = dz \cdot x$	-1.0000, -0.5000, -1.5000	-0.6416, -0.3208, -0.9625
6. $\partial L / \partial b = dz$	-0.5000	-0.3208
7. New $w \leftarrow w - 0.1 \cdot \text{grad}$	0.100, 0.050, 0.150	0.1642, 0.0821, 0.2462
8. New $b \leftarrow b - 0.1 \cdot \text{grad}$	0.050	0.0821

✓ WHAT JUST HAPPENED

Loss fell **0.6931 $\rightarrow$ 0.3869** (and 0.2561 on pass 3) while the prediction climbed **0.50 $\rightarrow$ 0.68 $\rightarrow$ 0.77**, closer to the truth $y = 1$ each time. Notice how simple the logistic gradient is: **$dz = \hat{y} - y$** , then multiply by each input — bigger input, bigger share of the blame.

JUMP TO:

1. Big picture + Linear Regression

2. Training loop + Logistic Regression

3. Confusions, overfitting, comparison

6 Four pairs beginners always mix up

ERROR vs LOSS

Error is for *one* example: $\hat{y} - y = 1.30 - 2.00 = -0.70$. It can be negative.
Loss is that error turned into one positive score, usually averaged over many examples (MSE = 0.49). *Error = the individual mistake. Loss = the overall report card.*

LOSS FUNCTION vs GRADIENT

Loss function tells you *how bad* you are — a height on a hill (0.49).
Gradient tells you *which way is downhill and how steep* — a direction (-1.40).
Loss = where you stand. Gradient = which way to walk.

BACKPROPAGATION vs OPTIMIZER

Backpropagation only *calculates* the gradients, passing blame backwards using the chain rule. It changes nothing.
Optimizer takes those gradients and actually *changes* the weights. *Backprop = the diagnosis. Optimizer = the treatment.*

GRADIENT DESCENT IS AN OPTIMIZER

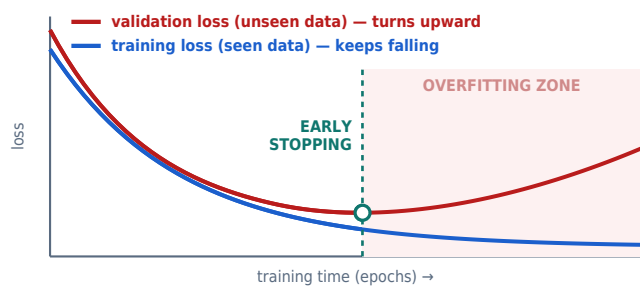
It is the simplest recipe for using gradients: $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \partial L / \partial \mathbf{w}$. η (learning rate) is the step size — too big and you leap over the valley, too small and you crawl. Adam and SGD are smarter versions of this same idea.

WHO DOES WHAT — the four words in order



7 Overfitting and how to stop it

OVERFITTING — memorising instead of understanding



Left of the green line the model is still learning **real patterns**. Right of it, training loss keeps falling but validation loss climbs — it has started **memorising noise**. That growing gap is overfitting.

Term	In plain words
Training loss	Score on the questions the model has already studied. Almost always keeps falling.
Validation loss	Score on unseen questions held back for testing. This is the honest score.
Generalisation	Doing well on data it has never seen — the real goal of every model.
Overfitting	Training loss low but validation loss high: it memorised the answer sheet.
Dropout	Randomly switch off some neurons each round so the model cannot lean on any one of them.
Early stopping	Stop training the moment validation loss starts to rise.
L2 regularisation	Add a penalty for large weights ($+\lambda \sum w^2$) so the model stays simple and smooth.

ANALOGY

A student who memorises last year's question paper scores 100% on it (low training loss) but fails the new paper (high validation loss). Dropout = hide random pages so they must truly understand. Early stopping = stop revising before cramming begins. L2 = reward simple, tidy answers over wild, extreme ones.

8 Final comparison — the one table to remember

What	Linear Regression	Logistic Regression
Question it answers	How much? How many?	Yes or no? Which class?
Output type	Any number ($-\infty$ to $+\infty$)	A probability between 0 and 1
Example	Predict flat price = ₹ 42,500	Will the customer buy? = 0.68 → Yes
Formula	$\hat{y} = w \cdot x + b$	$\hat{y} = \sigma(w \cdot x + b)$
Activation	None — raw sum is the answer	Sigmoid squashes the sum into 0–1
Loss function	MSE = mean of $(\hat{y} - y)^2$	BCE = $-[y \cdot \ln \hat{y} + (1 - y) \cdot \ln(1 - \hat{y})]$
Gradient signal	$\partial L / \partial w = 2(\hat{y} - y) \cdot x$	$\partial L / \partial w = (\hat{y} - y) \cdot x$ (even simpler!)
Decision step	None — use the number directly	Threshold: $\geq 0.5 \rightarrow$ class 1, else class 0
Optimizer	Gradient descent	Gradient descent
In a neural network	One neuron with no activation	One neuron with a sigmoid — the classic output neuron

THE 30-SECOND RECAP

Multiply inputs by weights, add bias, (optionally squash with sigmoid), compare with the truth to get a loss, let backpropagation compute the gradients, let gradient descent nudge every weight downhill — then repeat. That is all of it.